

T2 Modelos lineales

Índice

1. Introducción
2. Análisis discriminante cuadrático (QDA)
3. Análisis discriminante lineal (LDA)
4. Regresión logística binaria
5. Regresión logística
6. Regresión lineal

1 Introducción

Objetivo: introducir modelos lineales básicos

- Clasificadores generativos: QDA y LDA
- Clasificadores discriminativos: regresión logística binaria y regresión logística (multiclase)
- Regresión lineal

Clasificadores generativos: expresan **posteriors** en función de **priors** y **densidades condicionales** de las clases

$$p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) = \frac{p(\mathbf{x} \mid y = c, \boldsymbol{\theta}) p(y = c, \boldsymbol{\theta})}{\sum_{c'} p(\mathbf{x} \mid y = c', \boldsymbol{\theta}) p(y = c', \boldsymbol{\theta})} \propto p(\mathbf{x} \mid y = c, \boldsymbol{\theta}) p(y = c, \boldsymbol{\theta})$$

- **Análisis discriminante cuadrático (QDA):** modela las condicionales de clase con Gaussianas independientes
- **Análisis discriminante lineal (LDA):** modela las condicionales de clase con Gaussianas de Σ común

Ajuste de generativos: maximización de la **log-verosimilitud conjunta**, $\mathcal{L}(\boldsymbol{\theta}) = \sum_n \log p(\mathbf{x}_n, y_n \mid \boldsymbol{\theta})$

- **Sencillez:** los parámetros de cada clase suelen ajustarse separadamente, mediante promediado de datos de la clase
- **Flexibilidad:** pueden extenderse a nuevas clases sin necesidad de re-entrenar parámetros ya entrenados
- **Datos perdidos:** pueden tratarse fácilmente gracias al modelado de condicionales
- **Robustez:** frente a características degeneradas, también gracias al modelado de condicionales
- **Aprendizaje semi-supervisado:** facilitan el aprovechamiento de datos no etiquetados en aprendizaje

Clasificadores discriminativos: modelan **posteriors** directamente, sin necesidad de priors ni condicionales

- **Regresión logística binaria:** $C = 2$
- **Regresión logística (multiclase):** $C \geq 2$

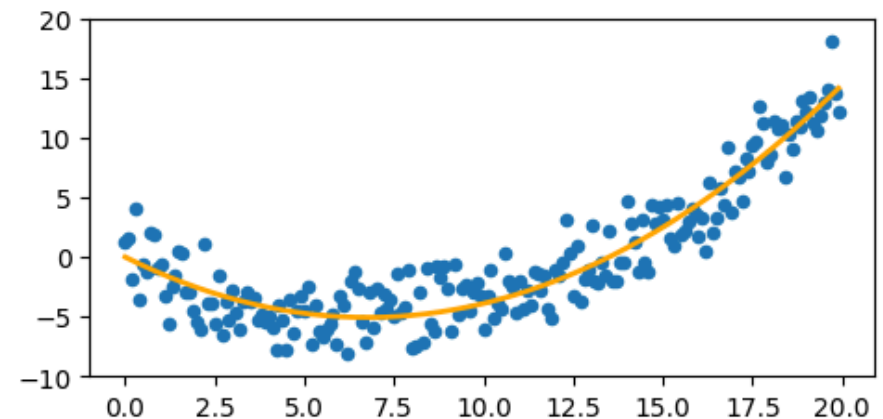
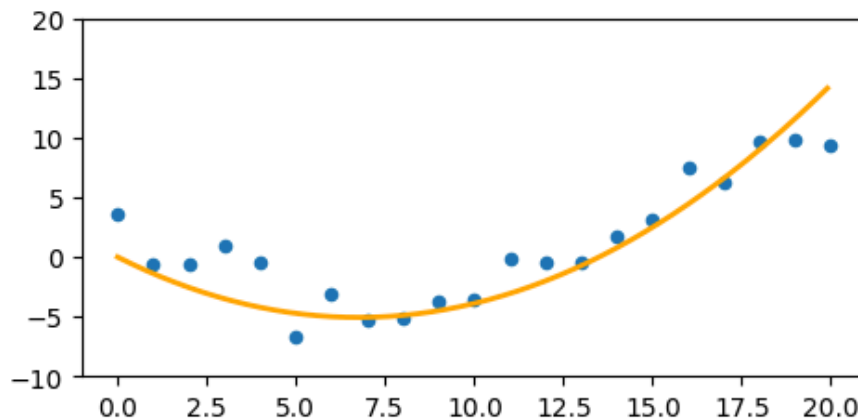
Ajuste de discriminativos: minimización de la neg-log-verosimilitud, $\mathcal{L}(\theta) = -\sum_n \log p(y_n | x_n, \theta)$

- **Precisión y calibrado de posteriors:** suelen ser más precisos que los generativos, con posteriors mejor calibradas
- **Ingeniería de características:** facilitan el preproceso de características, típicamente mediante expansión polinómica
- **Redes neuronales:** las redes usuales pueden verse como extensiones de modelos discriminativos básicos

Regresión lineal: regresión polinómica (de grado 2) simple con datos sintéticos de entrenamiento (izq.) y test (der.)

$$f(x; \mathbf{w}) = w_0 + w_1 x + w_2 x^2 \quad \text{con} \quad w_0 = 0, \quad w_1 = -1.5 \quad \text{y} \quad w_2 = 1/9 = 0.11; \quad \sigma^2 = 4$$

```
In [ ]: import numpy as np; np.random.seed(0); import matplotlib.pyplot as plt
w0 = 0; w1 = -1.5; w2 = 1/9; sigma = 2; f = lambda X: w0 + w1 * X + w2 * X * X
N = 21; X = np.linspace(0.0, 20, N); y = f(X) + np.random.normal(0, sigma, X.shape)
Xt = np.arange(0.0, 20, 0.1); yt = f(Xt) + np.random.normal(0, sigma, Xt.shape)
fig, axs = plt.subplots(1, 2, figsize=(12, 2.5))
ax = axs[0]; ax.set_ylim([-10, 20]); ax.scatter(X, y, s=20); ax.plot(Xt, f(Xt), c='orange', lw=2)
ax = axs[1]; ax.set_ylim([-10, 20]); ax.scatter(Xt, yt, s=20); ax.plot(Xt, f(Xt), c='orange', lw=2);
```



2 Análisis discriminante cuadrático (QDA)

Análisis discriminante Gaussiano (GDA): clasificador generativo de densidades condicionales Gaussianas

$$p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) \propto p(y = c, \boldsymbol{\theta}) p(\mathbf{x} \mid y = c, \boldsymbol{\theta}) = \pi_c \mathcal{N}_D(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

Función discriminante (en GDA): log-posterior de la clase vista como función de la entrada $\mathbf{x}$

$$\begin{aligned} \log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) &\propto \log \pi_c + \log \mathcal{N}_D(\mathbf{x} \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \\ &= \log \pi_c - \frac{D}{2} \log(2\pi) - \frac{1}{2} \log |\boldsymbol{\Sigma}_c| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_c)^t \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) \end{aligned}$$

Análisis discriminante cuadrático (QDA): GDA expresado en términos de discriminantes; cuadráticas con $\mathbf{x}$

$$\begin{aligned} \log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) &\propto \mathbf{x}^t \mathbf{A}_c \mathbf{x} + \boldsymbol{\beta}_c^t \mathbf{x} + \gamma_c + \kappa \\ \mathbf{A}_c &= -\frac{1}{2} \boldsymbol{\Sigma}_c^{-1} \\ \boldsymbol{\beta}_c &= \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c \\ \gamma_c &= \log \pi_c - \frac{1}{2} \log |\boldsymbol{\Sigma}_c| - \frac{1}{2} \boldsymbol{\mu}_c^t \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c \\ \kappa &= -\frac{D}{2} \log(2\pi) \quad (\text{puede ignorarse porque no depende de } c) \end{aligned}$$

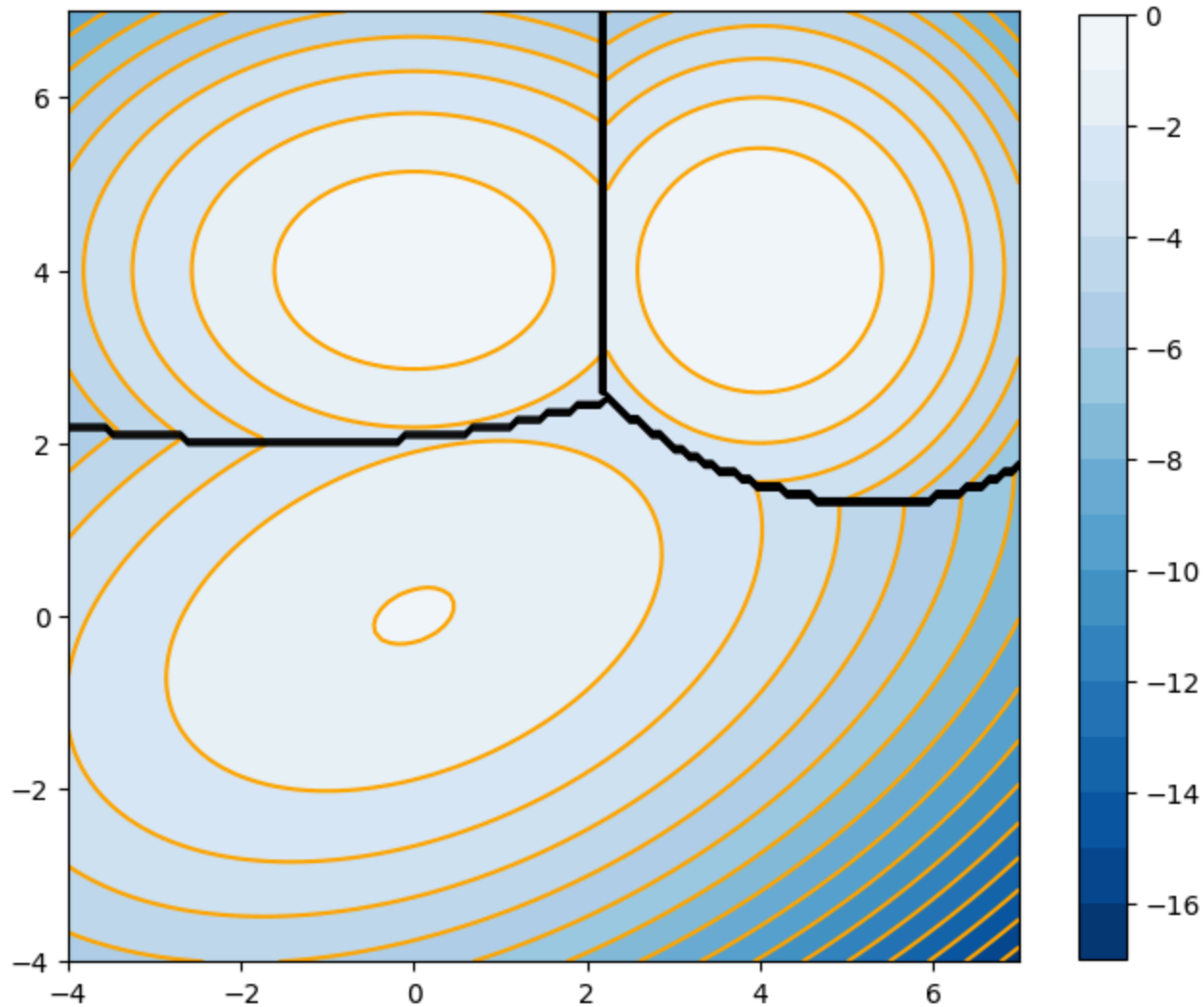
QDA diagonal: asume matrices de covarianzas diagonales; de $O(CD^2)$ a $O(CD)$ parámetros

Ejemplo: $C = 3$, $D = 2$, $\pi_1 = \pi_2 = \pi_3 = 1/3$ (podemos ignorar $\log \pi_c$ en γ_c)

$\mu_1 = (0, 0)^t$	$\mu_2 = (0, 4)^t$	$\mu_3 = (4, 4)^t$
$\Sigma_1 = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix}$	$\Sigma_2 = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$	$\Sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
$\Sigma_1^{-1} = \begin{pmatrix} 2/7 & -1/7 \\ -1/7 & 4/7 \end{pmatrix}$	$\Sigma_2^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1 \end{pmatrix}$	$\Sigma_3^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
$A_1 = \begin{pmatrix} -1/7 & 1/14 \\ 1/14 & -2/7 \end{pmatrix}$	$A_2 = \begin{pmatrix} -1/4 & 0 \\ 0 & -1/2 \end{pmatrix}$	$A_3 = \begin{pmatrix} -1/2 & 0 \\ 0 & -1/2 \end{pmatrix}$
$\beta_1 = (0, 0)^t$	$\beta_2 = (0, 4)^t$	$\beta_3 = (4, 4)^t$
$\gamma_1 = -\frac{1}{2} \log 7 = -0.973$	$\gamma_2 = -\frac{1}{2} \log 2 - 8 = -8.3466$	$\gamma_3 = -16$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
A1, b1, c1 = np.array([ [-1/7, 1/14], [1/14, -2/7] ]), np.array([0, 0]), -0.973
A2, b2, c2 = np.array([ [-1/4, 0], [0, -1/2] ]), np.array([0, 4]), -8.3466
A3, b3, c3 = np.array([ [-1/2, 0], [0, -1/2] ]), np.array([4, 4]), -16
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 = lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 = lambda x: x.T @ A2 @ x + b2 @ x + c2
p3 = lambda x: x.T @ A3 @ x + b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x))
maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)])
amaxpx = np.apply_along_axis(amaxp, 1, x);
```

```
In [ ]: fig, ax = plt.subplots(1, 1, figsize=(8, 8)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyle='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8);
```



Log-verosimilitud conjunta: $\boldsymbol{\theta}' = \{\boldsymbol{\theta}_c\}$, $\boldsymbol{\theta}_c = (\boldsymbol{\mu}_c, \text{vec}(\boldsymbol{\Sigma}_c))^t$

$$\log p(\mathcal{D} \mid \boldsymbol{\theta}) = \sum_c N_c \log \pi_c + \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

Ajuste de $\boldsymbol{\theta}'$: $\hat{\boldsymbol{\theta}}'$ puede hallarse mediante maximización separada en cada $\boldsymbol{\theta}_c$ (con restricciones)

$$\begin{aligned} \hat{\boldsymbol{\theta}}' &= \underset{\boldsymbol{\theta}' \in \mathcal{C}_{\boldsymbol{\theta}'}}{\operatorname{argmax}} \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \\ &\quad \Updownarrow \\ \hat{\boldsymbol{\theta}}_c &= \underset{\boldsymbol{\theta}_c \in \mathcal{C}_{\boldsymbol{\theta}_c}}{\operatorname{argmax}} \sum_{n:y_n=c} \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \quad \text{para todo } c \end{aligned}$$

Ajuste de $\boldsymbol{\theta}'_c$: $\hat{\boldsymbol{\mu}}_c$ y $\hat{\boldsymbol{\Sigma}}_c$ se hallan con los datos de la clase c como el MLE de una única Gaussiana

$$\begin{aligned} \hat{\boldsymbol{\mu}}_c &= \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n \\ \hat{\boldsymbol{\Sigma}}_c &= \frac{1}{N_c} \sum_{n:y_n=c} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)^t \end{aligned}$$

Ejemplo (cont.): $C = 3$, $D = 2$, $\pi_1 = \pi_2 = \pi_3 = 1/3$

$$\begin{aligned} \mu_1 &= (0, 0)^t & \mu_2 &= (0, 4)^t & \mu_3 &= (4, 4)^t \\ \Sigma_1 &= \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix} & \Sigma_2 &= \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} & \Sigma_3 &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \end{aligned}$$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import multinomial, multivariate_normal
N = 50 # >=3 para tener al menos un dato por clase; >>3 para evitar matrices singulares
pi1 = pi2 = pi3 = 1/3
yy = multinomial(1, [pi1, pi2, pi3]).rvs(N - 3)
N1 = yy[yy[:, 0] == 1].shape[0] + 1
N2 = yy[yy[:, 1] == 1].shape[0] + 1
N3 = N - N1 - N2
hpi1 = N1/N; hpi2 = N2/N; hpi3 = N3/N
xxy1 = multivariate_normal([0, 0], [[4, 1], [1, 2]]).rvs(N1)
m1 = xxy1.mean(axis=0); S1 = np.cov(xxy1, rowvar=False, bias=True); iS1 = np.linalg.inv(S1)
xxy2 = multivariate_normal([0, 4], [[2, 0], [0, 1]]).rvs(N2)
m2 = xxy2.mean(axis=0); S2 = np.cov(xxy2, rowvar=False, bias=True); iS2 = np.linalg.inv(S2)
xxy3 = multivariate_normal([4, 4], np.eye(2)).rvs(N3)
m3 = xxy3.mean(axis=0); S3 = np.cov(xxy3, rowvar=False, bias=True); iS3 = np.linalg.inv(S3)
print("Medias: ", m1, m2, m3, "\nSigmas:\n", S1, "\n", S2, "\n", S3)
```

Medias: [0.33956314 0.65842643] [-0.00803341 3.56576981] [3.83229691 4.00927197]

Sigmas:

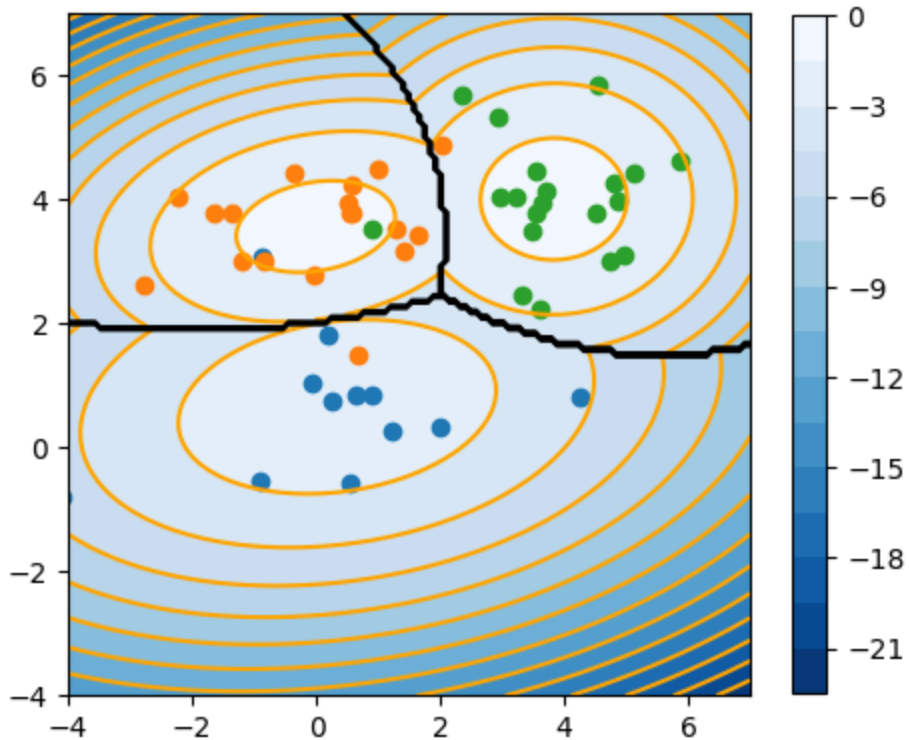
```
[[3.53223658 0.36163145]
 [0.36163145 1.06158044]]
[[1.82163006 0.21941638]
 [0.21941638 0.60543893]]
[[1.20967908 0.00839126]
 [0.00839126 0.83738662]]
```



```

In [ ]: A1, b1, c1 = -0.5 * iS1, iS1 @ m1, np.log(hpi1) - 0.5 * np.log(np.linalg.det(S1)) - 0.5 * m1.T @ iS1 @ m1
A2, b2, c2 = -0.5 * iS2, iS2 @ m2, np.log(hpi2) - 0.5 * np.log(np.linalg.det(S2)) - 0.5 * m2.T @ iS2 @ m2
A3, b3, c3 = -0.5 * iS3, iS3 @ m3, np.log(hpi3) - 0.5 * np.log(np.linalg.det(S3)) - 0.5 * m3.T @ iS3 @ m3
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]; p1 = lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 = lambda x: x.T @ A2 @ x + b2 @ x + c2; p3 = lambda x: x.T @ A3 @ x + b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x)); maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)]); amaxpx = np.apply_along_axis(amaxp, 1, x)
fig, ax = plt.subplots(1, 1, figsize=(5.5, 5.5)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyle='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8); ax.set_xlim(-4, 7); ax.set_ylim(-4, 7)
plt.scatter(xxy1[:, 0], xxy1[:, 1]); plt.scatter(xxy2[:, 0], xxy2[:, 1])
plt.scatter(xxy3[:, 0], xxy3[:, 1]);

```



3 Análisis discriminante lineal (LDA)

Análisis discriminante lineal (LDA): GDA con Gaussianas de **matriz de covarianza común**, $\Sigma_c = \Sigma$

$$\log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) \propto \log \pi_c + \log \mathcal{N}_D(\mathbf{x} \mid \boldsymbol{\mu}_c, \Sigma)$$

Discriminantes lineales con $\mathbf{x}$: al igual que κ , el término cuadrático puede ignorarse porque no depende de c

$$\log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) \propto \log \pi_c - \frac{D}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_c)^t \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}_c)$$

$$= \mathbf{x}^t \mathbf{A} \mathbf{x} + \boldsymbol{\beta}_c^t \mathbf{x} + \gamma_c + \kappa$$

$$\mathbf{A} = -\frac{1}{2} \Sigma^{-1}$$

$$\boldsymbol{\beta}_c = \Sigma^{-1} \boldsymbol{\mu}_c$$

$$\gamma_c = \log \pi_c - \frac{1}{2} \boldsymbol{\mu}_c^t \Sigma^{-1} \boldsymbol{\mu}_c$$

$$\kappa = -\frac{D}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma|$$

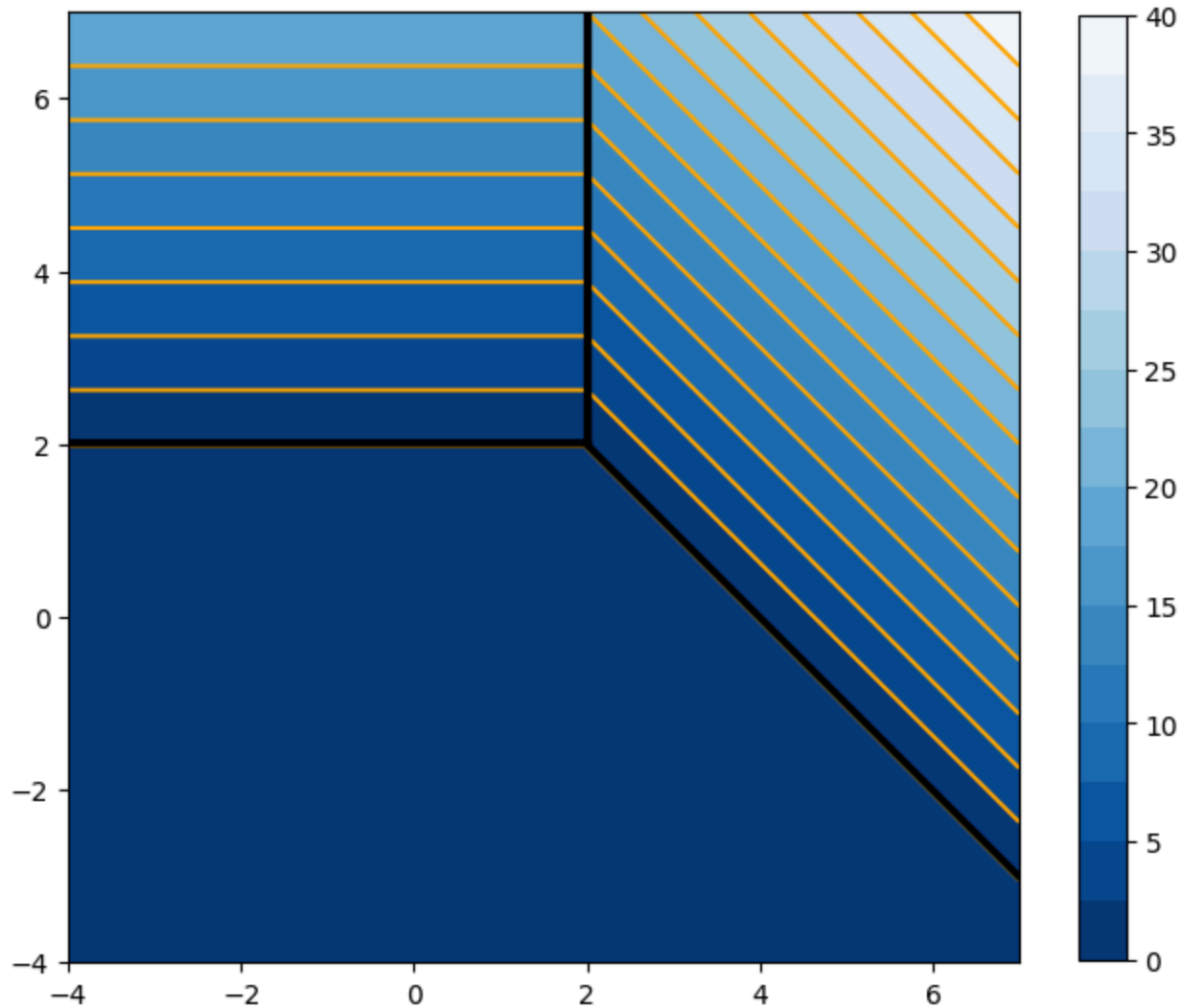
LDA diagonal: asume Σ diagonal; de $O(CD + D^2)$ a $O(CD)$ parámetros

Ejemplo: $C = 3$, $D = 2$, $\pi_1 = \pi_2 = \pi_3 = 1/3$ (podemos ignorar $\log \pi_c$ en γ_c)

$$\begin{array}{lll} \boldsymbol{\mu}_1 = (0, 0)^t & \boldsymbol{\mu}_2 = (0, 4)^t & \boldsymbol{\mu}_3 = (4, 4)^t \\ \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} & \boldsymbol{\Sigma}^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} & \\ \boldsymbol{\beta}_1 = (0, 0)^t & \boldsymbol{\beta}_2 = (0, 4)^t & \boldsymbol{\beta}_3 = (4, 4)^t \\ \gamma_1 = 0 & \gamma_2 = -8 & \gamma_3 = -16 \end{array}$$

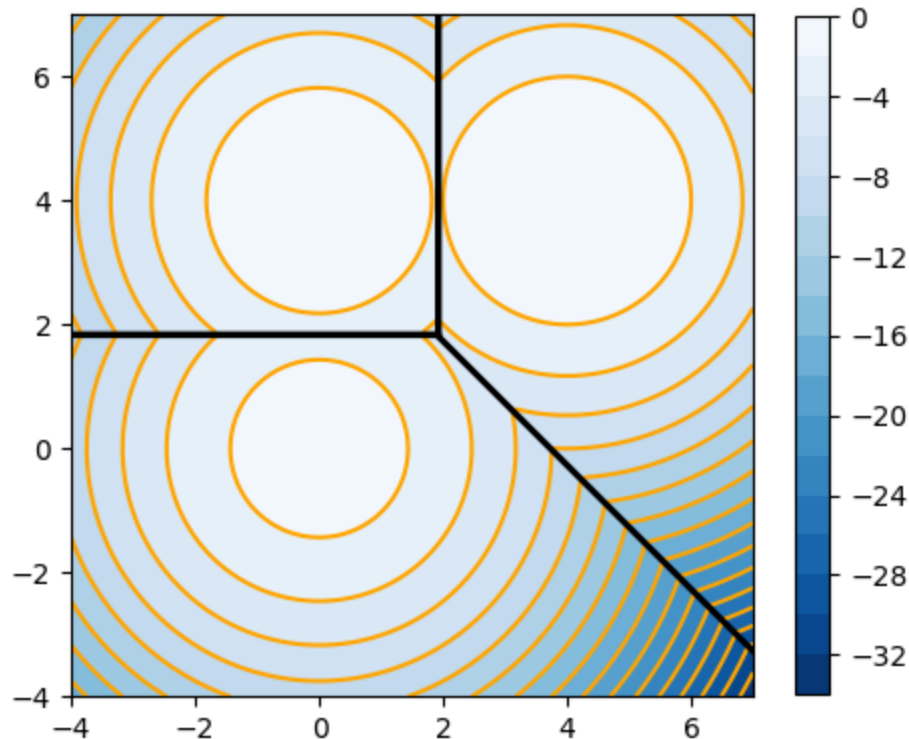
```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
b1, c1 = np.array([0, 0]), 0
b2, c2 = np.array([0, 4]), -8
b3, c3 = np.array([4, 4]), -16
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 = lambda x: b1 @ x + c1
p2 = lambda x: b2 @ x + c2
p3 = lambda x: b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x))
maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)])
amaxpx = np.apply_along_axis(amaxp, 1, x)
```

```
In [ ]: fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyle='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8);
```



Los conjuntos de nivel típicos de las Gaussianas pueden obtenerse añadiendo el término cuadrático común a las discriminantes, si bien el mismo es irrelevante a efectos de regiones y fronteras de decisión:

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
A1, b1, c1 = np.array([ [-1/2, 0], [ 0, -1/2] ]), np.array([0, 0]), -0.973
A2, b2, c2 = np.array([ [-1/2, 0], [ 0, -1/2] ]), np.array([0, 4]), -8.3466
A3, b3, c3 = np.array([ [-1/2, 0], [ 0, -1/2] ]), np.array([4, 4]), -16
R = np.linspace(-4, 7, 128); x1, x2 = np.meshgrid(R, R); x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 = lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 = lambda x: x.T @ A2 @ x + b2 @ x + c2
p3 = lambda x: x.T @ A3 @ x + b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x)); maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)]); amaxpx = np.apply_along_axis(amaxp, 1, x)
fig, ax = plt.subplots(1, 1, figsize=(5.5, 5.5)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyle='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8);
```



Log-verosimilitud conjunta: $\boldsymbol{\theta}' = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_C, \text{vec}(\boldsymbol{\Sigma}))^t$

$$\log p(\mathcal{D} \mid \boldsymbol{\theta}) = \sum_c N_c \log \pi_c + \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma})$$

Ajuste de $\boldsymbol{\theta}'$: no puede hacerse clase a clase, separadamente, porque las condicionales comparten una misma $\boldsymbol{\Sigma}$

$$\hat{\boldsymbol{\theta}}' = \underset{\boldsymbol{\theta}' \in \mathcal{C}_{\boldsymbol{\theta}'}}{\operatorname{argmax}} \sum_c \sum_{n:y_n=c} \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_c, \boldsymbol{\Sigma})$$

No obstante, la única diferencia respecto a QDA es que $\hat{\boldsymbol{\Sigma}}$ se obtiene con todos los datos:

$$\begin{aligned}\hat{\boldsymbol{\mu}}_c &= \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n \\ \hat{\boldsymbol{\Sigma}} &= \frac{1}{N} \sum_c \sum_{n:y_n=c} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)^t\end{aligned}$$

Ejemplo (cont.): $C = 3$, $D = 2$, $\pi_1 = \pi_2 = \pi_3 = 1/3$

$$\begin{aligned}\mu_1 &= (0, 0)^t & \mu_2 &= (0, 4)^t & \mu_3 &= (4, 4)^t \\ \Sigma &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} & \Sigma^{-1} &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\end{aligned}$$

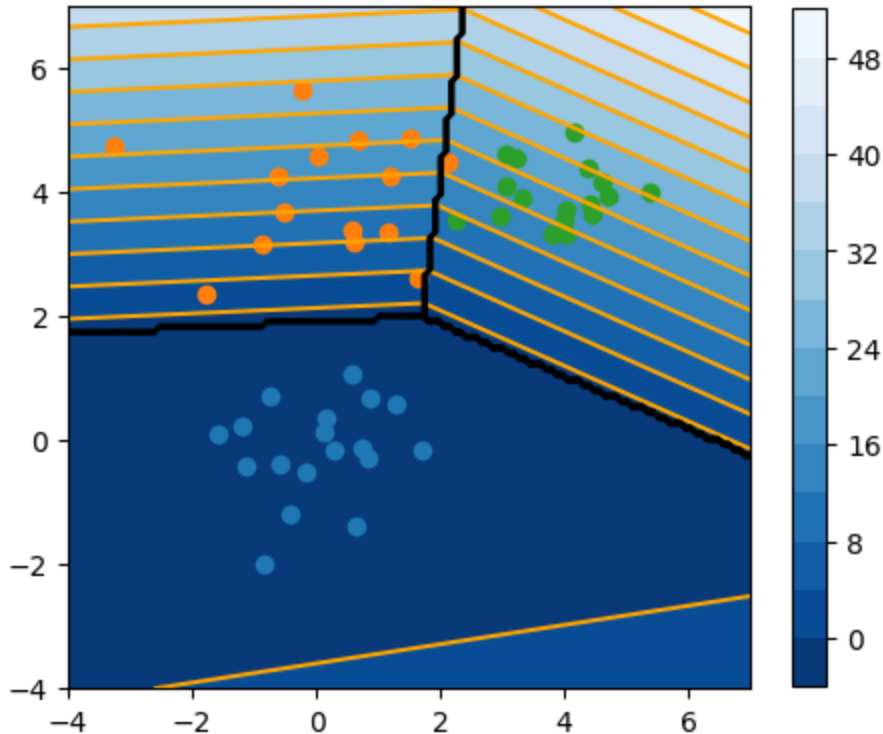
```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from scipy.stats import multinomial, multivariate_normal
N = 50 # >=3 para tener al menos un dato por clase; >>3 para evitar matrices singulares
pi1 = pi2 = pi3 = 1/3
yy = multinomial(1, [pi1, pi2, pi3]).rvs(N - 3)
N1 = yy[yy[:, 0] == 1].shape[0] + 1
N2 = yy[yy[:, 1] == 1].shape[0] + 1
N3 = N - N1 - N2
hpi1 = N1/N; hpi2 = N2/N; hpi3 = N3/N
xxy1 = multivariate_normal([0, 0], np.eye(2)).rvs(N1)
m1 = xxy1.mean(axis=0); S1 = np.cov(xxy1, rowvar=False, bias=True)
xxy2 = multivariate_normal([0, 4], np.eye(2)).rvs(N2)
m2 = xxy2.mean(axis=0); S2 = np.cov(xxy2, rowvar=False, bias=True)
xxy3 = multivariate_normal([4, 4], np.eye(2)).rvs(N3)
m3 = xxy3.mean(axis=0); S3 = np.cov(xxy3, rowvar=False, bias=True)
S = hpi1 * S1 + hpi2 * S2 + hpi3 * S3; iS = np.linalg.inv(S)
print("Medias: ", m1, m2, m3, "\nSigma:\n", S)
```

```
Medias: [ 0.02852604 -0.14860485] [0.14946012 3.96905472] [3.87641211 3.95226088]
Sigma:
[[1.06255583 0.06651106]
 [0.06651106 0.52196998]]
```

```

In [ ]: b1, c1 = iS @ m1, np.log(hpi1) - 0.5 * m1.T @ iS @ m1
b2, c2 = iS @ m2, np.log(hpi2) - 0.5 * m2.T @ iS @ m2
b3, c3 = iS @ m3, np.log(hpi3) - 0.5 * m3.T @ iS @ m3
x1, x2 = np.meshgrid(np.linspace(-4, 7, num=128), np.linspace(-4, 7, num=128))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 = lambda x: b1 @ x + c1; p2 = lambda x: b2 @ x + c2; p3 = lambda x: b3 @ x + c3
maxp = lambda x: max(p1(x), p2(x), p3(x)); maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x), p3(x)]); amaxpx = np.apply_along_axis(amaxp, 1, x)
fig, ax = plt.subplots(1, 1, figsize=(5.5, 5.5)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='black', linestyle='solid', linewidths=1)
plt.colorbar(cp, ax=ax, shrink=0.8); ax.set_xlim(-4, 7); ax.set_ylim(-4, 7)
plt.scatter(xxy1[:, 0], xxy1[:, 1]); plt.scatter(xxy2[:, 0], xxy2[:, 1])
plt.scatter(xxy3[:, 0], xxy3[:, 1]);

```



Clasificador por el centroide más próximo (NCM): regla de decisión MAP para LDA con priors idénticos,

$$\begin{aligned}\hat{y}(\mathbf{x}) &= \operatorname{argmax}_c \log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) \\ &= \operatorname{argmax}_c \log p(\mathbf{x} \mid y = c, \boldsymbol{\theta}) && \text{(asumiendo priors idénticos)} \\ &= \operatorname{argmax}_c -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^t \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_c) \\ &= \operatorname{argmin}_c (\mathbf{x} - \boldsymbol{\mu}_c)^t \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_c) \\ &= \operatorname{argmin}_c \|\mathbf{x} - \boldsymbol{\mu}_c\|_{\mathbf{W}}^2 && \text{(con } \boldsymbol{\Sigma}^{-1} = \mathbf{W}^t \mathbf{W}, \|\mathbf{z}\|_{\mathbf{W}}^2 = \mathbf{z}^t \mathbf{W}^t \mathbf{W} \mathbf{z})\end{aligned}$$

Motivación: aparte de su sencillez y reducido coste computacional, puede usarse en **one-shot learning** de nuevas clases ya que basta añadir un prototipo etiquetado $\boldsymbol{\mu}_c$ por cada clase c nueva que se quiera incluir

Aprendizaje de la métrica: $\mathbf{W}$ puede ser una raíz de $\boldsymbol{\Sigma}^{-1}$ o aprenderse con algún otro criterio

4 Regresión logística binaria

Regresión logística binaria: Bernoulli condicional para clasificación binaria, $y \in \{0, 1\}$,

$$p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(y \mid \mu) \quad \text{con} \quad \mu = \sigma(a) \quad \text{y log-odds} \quad a = f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{w}^t \mathbf{x} + b$$

$$p(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

$$p(y = 0 \mid \mathbf{x}; \boldsymbol{\theta}) = 1 - \sigma(a) = \sigma(-a) = \frac{1}{1 + e^a}$$

Con etiquetas $\tilde{y} \in \{-1, 1\}$: $p(\tilde{y} \mid \mathbf{x}; \boldsymbol{\theta}) = \sigma(\tilde{y}a)$

Regla de decisión MAP: $\mathbf{x}$ se clasifica en la clase 1 si la log-odds es positiva

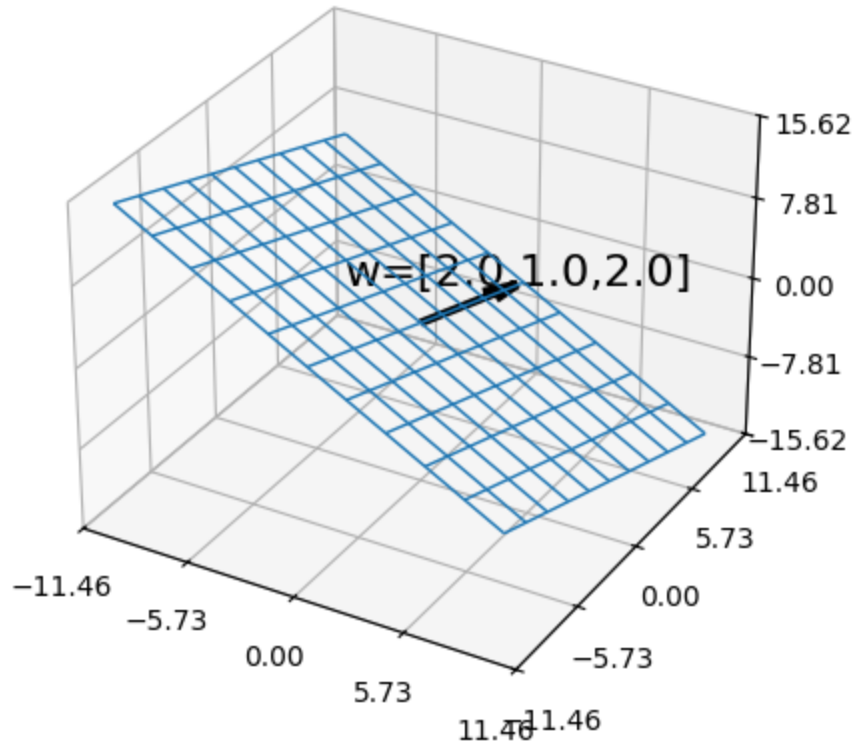
$$\mathbb{I}(p(y = 1 \mid \mathbf{x}) > p(y = 0 \mid \mathbf{x})) = \mathbb{I}\left(\log \frac{p(y = 1 \mid \mathbf{x})}{p(y = 0 \mid \mathbf{x})} > 0\right) = \mathbb{I}(a > 0)$$

Frontera MAP hiperplanar: $\mathbf{w}^t \mathbf{x} + b = 0$

Notación homogénea: asume sesgo absorbido en $\mathbf{w}$ (con la ayuda de una dimensión de entrada extra e igual a 1)

Ejemplo de frontera: $w = (2, 1, 2)^t$ y $b = 0$; frontera $2x_1 + x_2 + 2x_3 + 0 = 0$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
w1, w2, w3, b = 2.0, 1.0, 2.0, 0.0
x1, x2 = np.meshgrid(np.linspace(-10, 10, 11), np.linspace(-10, 10, 11))
x3 = lambda x1, x2: (-w1 * x1 - w2 * x2 - b) / w3
fig = plt.figure(figsize=(5, 5)); ax = fig.add_subplot(111, projection='3d')
ax.plot_wireframe(x1, x2, x3(x1, x2), rstride=1, cstride=1, linewidth=1)
scaw = 2.0; ax.quiver(0, 0, x3(0, 0), scaw * w1, scaw * w2, scaw * w3, linewidth=3, colors='black')
ax.text(scaw * w1, scaw * w2, scaw * w3, f"w=[{w1},{w2},{w3}]", fontsize=16, ha='center')
x_min, x_max = ax.get_xlim(); ax.set_xticks(np.linspace(x_min, x_max, 5))
y_min, y_max = ax.get_ylim(); ax.set_yticks(np.linspace(y_min, y_max, 5))
z_min, z_max = ax.get_zlim(); ax.set_zticks(np.linspace(z_min, z_max, 5));
```



Neg-log-verosimilitud (condicional): de $\mathbf{w}$ respecto a N datos $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}$ (normalizada por N)

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N \log p(y_n | \mathbf{x}_n, \mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)$$

Gradiente (transpuesto): con $\mathbf{y} = (y_1, \dots, y_N)^t$

$$\frac{\partial \text{NLL}(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \sum_n (\mu_n - y_n) \mathbf{x}_n^t = \frac{1}{N} (\boldsymbol{\sigma}(\mathbf{X}\mathbf{w}) - \mathbf{y})^t \mathbf{X}$$

Descenso por gradiente:

$$\begin{aligned} \mathbf{w}_{i+1} &= \mathbf{w}_i - \eta_i \frac{1}{N} \mathbf{X}^t (\boldsymbol{\sigma}(\mathbf{X}\mathbf{w}_i) - \mathbf{y}) \\ \mathbf{w}_{i+1} &= \mathbf{w}_i - \eta_i \mathbf{x}_n (\sigma(\mathbf{x}_n^t \mathbf{w}_i) - y_n) \quad (\text{para batch unitario}) \end{aligned}$$

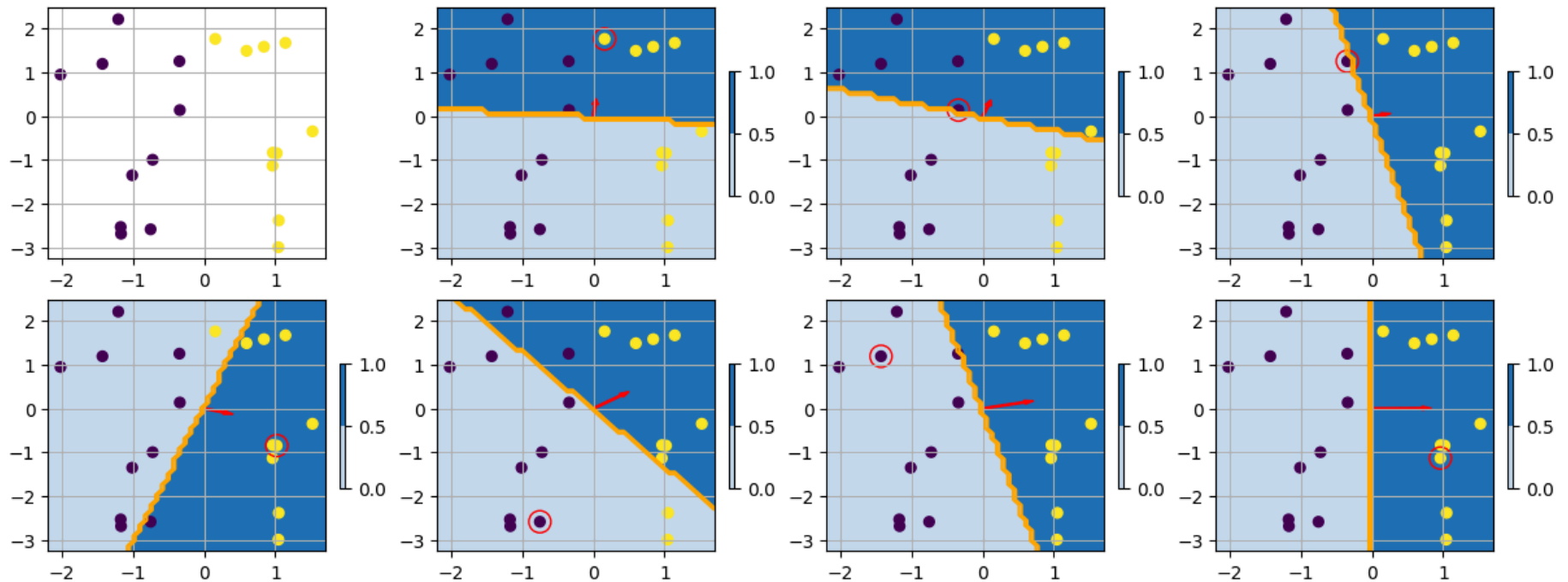
Ejemplo: $\mathbf{X} = [1, 1, 0; 1, 0, 1; 1, -1, 0; 1, 0, -1]$ y $\mathbf{y} = (0, 1, 1, 0)$ con sesgo absorbido en dimensión 1

```
In [ ]: import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,1,0],[1,0,1],[1,-1,0],[1,0,-1]]); y = np.array([0,1,1,0]); eta = .01; w = np.zeros(3)
for i in range(100):
    w = w - eta / len(y) * X.T@(1/(1+np.exp(-X@w))-y)
print(w)
```

```
[0.00 -0.24 0.24]
```

Ejemplo: datos sintéticos 2d y modelo sin sesgo; w normal a la frontera, apunta a la clase 1

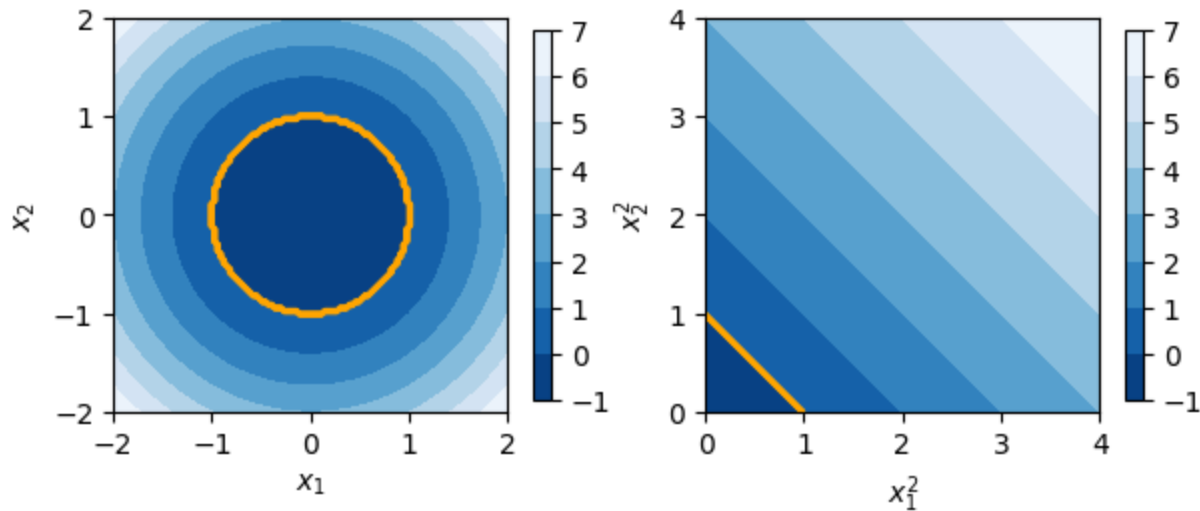
```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from sklearn.datasets import make_classification
N, n_clusters_per_class, class_sep = 20, 2, 1.0
X, y = make_classification(n_samples=N, n_features=2, n_redundant=0, n_classes=2,
    n_clusters_per_class=n_clusters_per_class, class_sep=class_sep, random_state=1)
nrows = 2; ncols = 4; fig, axs = plt.subplots(nrows, ncols, figsize=(12, 4.5), constrained_layout=True)
ax = axs.flat[0]; ax.grid(); ax.scatter(*X.T, c=y, s=32)
x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 50), np.linspace(y_min, y_max, 50))
XX = np.c_[np.ravel(xx), np.ravel(yy)]; w = np.zeros(2); eta = 0.3
for n in np.arange(1, min(N, nrows * ncols)):
    mun = 1.0 / (1.0 + np.exp(- w @ X[n, :])); w = w - eta * X[n, :] * (mun - y[n])
    ax = axs.flat[n]; ax.grid()
    z = lambda x: w @ x; zz = np.heaviside(np.apply_along_axis(z, 1, XX), 0.0)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyle='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 1, cmap='Blues'); plt.colorbar(cp, ax=ax, shrink=0.5)
    ax.arrow(0, 0, *w, width=.03, facecolor='red', edgecolor='red')
    ax.scatter(*X.T, c=y, s=32); ax.scatter(X[n, 0], X[n, 1], facecolors='none', edgecolors='red', s=150)
```



Ingeniería de características: lineariza un problema con datos no linealmente separables mediante una función $\phi(\mathbf{x})$

Ejemplo: $f(\mathbf{x}) = x_1^2 + x_2^2 - R^2 = \mathbf{w}^t \phi(\mathbf{x}) + b$ con $\phi(x_1, x_2) = (x_1^2, x_2^2)$, $\mathbf{w} = (1, 1)$ y $b = -R^2$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; R = 1
xr = np.linspace(-2, 2, 128); x1, x2 = np.meshgrid(xr, xr); X = np.c_[np.ravel(x1), np.ravel(x2)]
z = lambda x: x[0]**2 + x[1]**2 - R**2; Z = np.apply_along_axis(z, 1, X)
fig, axes = plt.subplots(1, 2, figsize=(7, 3)); axes[0].set(aspect='equal')
axes[0].set_xlabel('$x_1$', fontsize=10); axes[0].set_ylabel('$x_2$', fontsize=10)
axes[0].contour(x1, x2, (Z > 0).reshape(x1.shape), 8, colors='orange', linestyle='solid')
cp = axes[0].contourf(x1, x2, Z.reshape(x1.shape), 8, cmap='Blues_r')
plt.colorbar(cp, ax=axes[0], shrink=0.8); xxr = np.linspace(0, 4, 128)
xx1, xx2 = np.meshgrid(xxr, xxr); XX = np.c_[np.ravel(xx1), np.ravel(xx2)]
zz = lambda xx: xx[0] + xx[1] - R**2; ZZ = np.apply_along_axis(zz, 1, XX); axes[1].set(aspect='equal')
axes[1].set_xlabel('$x_1^2$', fontsize=10); axes[1].set_ylabel('$x_2^2$', fontsize=10)
axes[1].contour(xx1, xx2, (ZZ > 0).reshape(xx1.shape), 8, colors='orange', linestyle='solid')
cp = axes[1].contourf(xx1, xx2, ZZ.reshape(xx1.shape), 8, cmap='Blues_r')
plt.colorbar(cp, ax=axes[1], shrink=0.8);
```



5 Regresión logística

Regresión logística: categórica condicional para clasificación multiclase, $y \in \{1, \dots, C\}$,

$$p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \text{Cat}(y \mid \boldsymbol{\mu}), \quad \boldsymbol{\mu} = \mathcal{S}(\mathbf{a}), \quad \text{logits } \mathbf{a} = f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}^t \mathbf{x} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{D \times C}, \mathbf{b} \in \mathbb{R}^C$$

Regla de decisión MAP: $\mathbf{x}$ se clasifica en la clase c si su logit es mayor que el resto

$$p(y = c \mid \mathbf{x}) > \max_{c' \neq c} p(y = c' \mid \mathbf{x}) \quad \Leftrightarrow \quad a_c > a_{c'} \quad \text{para todo } c' \neq c$$

Región de decisión: intersección de $C - 1$ semi-espacios, esto es, un poliedro D -dimensional (acotado o no)

$$\mathcal{R}_c = \bigcup_{c' \neq c} \{\mathbf{x} : a_c > a_{c'}\} = \bigcup_{c' \neq c} \{\mathbf{x} : (\mathbf{W}_{:,c})^t \mathbf{x} > (\mathbf{W}_{:,c'})^t \mathbf{x}\}$$

Notación homogénea: asume sesgo absorbido en $\mathbf{W}$ (con la ayuda de una dimensión de entrada extra e igual a 1)

Neg-log-verosimilitud (condicional): de $\mathbf{W}$ respecto a N datos $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}$ con etiquetas de clase one-hot

$$\text{NLL}(\mathbf{W}) = -\frac{1}{N} \sum_{n=1}^N \log p(\mathbf{y}_n | \mathbf{x}_n, \mathbf{W}) = -\frac{1}{N} \sum_{n=1}^N \sum_{c=1}^C y_{nc} \log \mu_{nc}$$

Gradiente (transpuesto): con $\mathcal{S}$ aplicada por filas, $\mathbf{Y} = [\mathbf{y}_1, \dots, \mathbf{y}_N]^t$

$$\frac{\partial \text{NLL}(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{N} \sum_{n=1}^N (\boldsymbol{\mu}_n - \mathbf{y}_n) \mathbf{x}_n^t = \frac{1}{N} (\mathcal{S}(\mathbf{XW}) - \mathbf{Y})^t \mathbf{X}$$

Descenso por gradiente:

$$\begin{aligned} \mathbf{W}_{i+1} &= \mathbf{W}_i - \eta_i \frac{1}{N} \mathbf{X}^t (\mathcal{S}(\mathbf{XW}_i) - \mathbf{Y}) \\ \mathbf{W}_{i+1} &= \mathbf{W}_i - \eta_i \mathbf{x}_n (\mathcal{S}(\mathbf{x}_n^t \mathbf{W}_i) - \mathbf{y}_n^t) \quad (\text{para batch unitario}) \end{aligned}$$

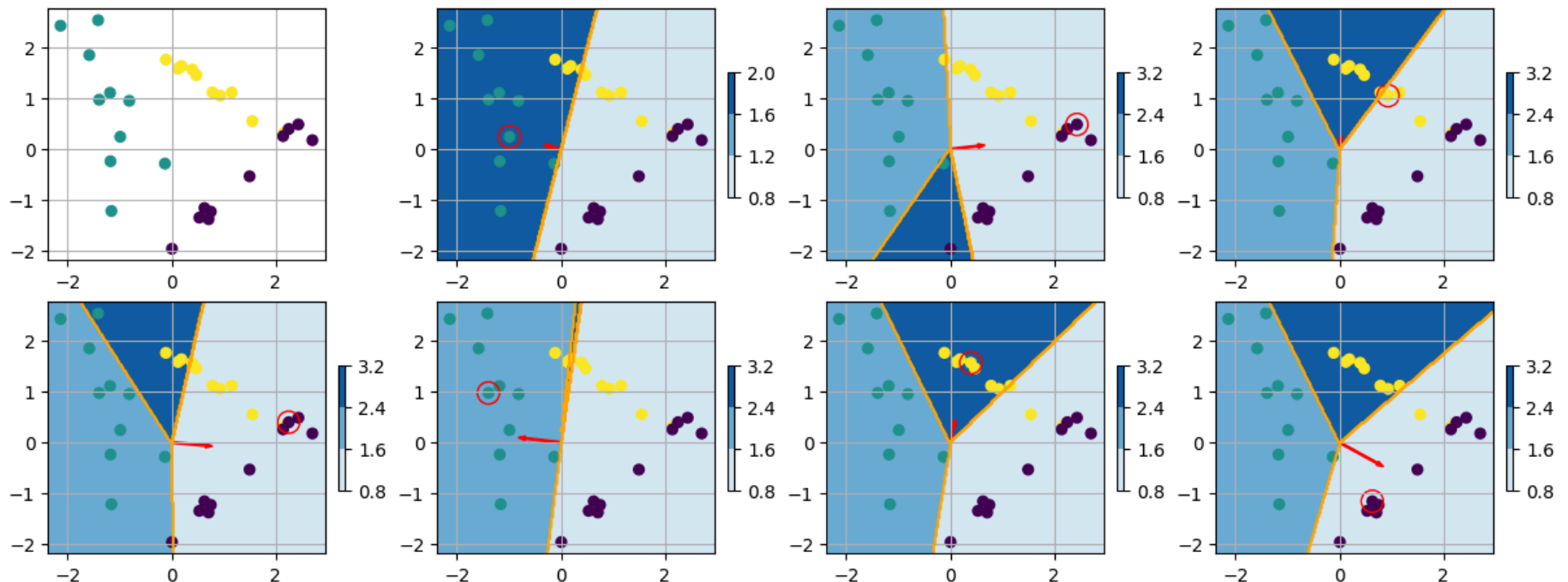
Ejemplo: $\mathbf{X} = [1, 1, 1; 1, -1, 1; 1, -1, -1; 1, 1, -1]$ y $\mathbf{y} = (1, 2, 3, 4)$; obtén $\hat{\mathbf{W}}$ con descenso por gradiente

```
In [ ]: import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.4f}".format(x)})
X = np.array([[1,1,1],[1,-1,1],[1,-1,-1],[1,1,-1]]); N = X.shape[0]
Y = np.array([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1]]); eta = .01; W = np.zeros((3, 4))
for i in range(1000):
    Z = np.apply_along_axis(np.exp, 1, X @ W); Z = Z/Z.sum(axis=1, keepdims=True); Z -= Y
    W = W - eta / N * X.T@Z
print(W)
```

```
[[-0.0000 -0.0000 0.0000 -0.0000]
 [1.0891 -1.0891 -1.0891 1.0891]
 [1.0891 1.0891 -1.0891 -1.0891]]
```

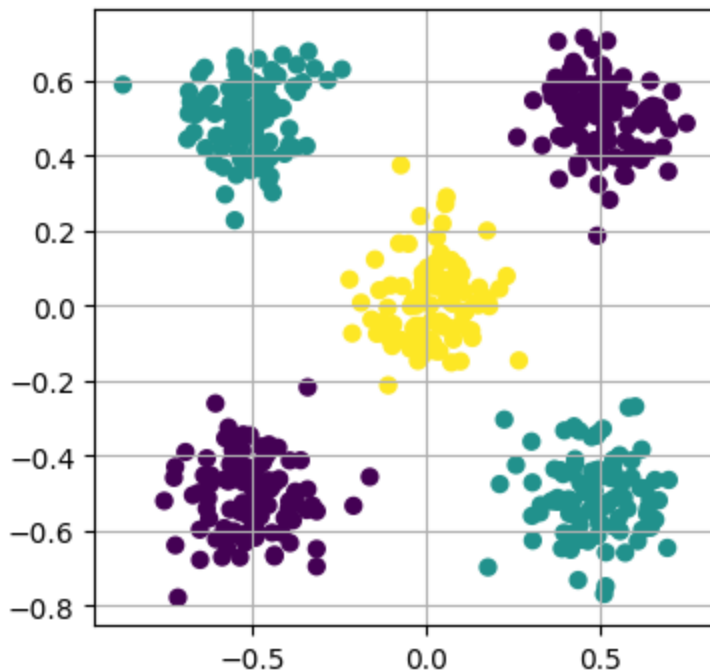

Ejemplo: datos sintéticos 2d y modelo sin sesgo; cada columna de W apunta a su clase

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; from sklearn.datasets import make_classification
from scipy.special import logsumexp; N, n_clusters_per_class, class_sep = 30, 1, 1.0
X, y = make_classification(n_samples=N, n_features=2, n_redundant=0, n_classes=3,
    n_clusters_per_class=n_clusters_per_class, class_sep=class_sep, random_state=43)
nrows = 2; ncols = 4; fig, axs = plt.subplots(nrows, ncols, figsize=(12, 4.5), constrained_layout=True)
ax = axs.flat[0]; ax.grid(); ax.scatter(*X.T, c=y, s=32)
x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
XX = np.c_[np.ravel(xx), np.ravel(yy)]; W = np.zeros((2, 3)); eta = 0.3
for n in np.arange(1, min(N, nrows * ncols)):
    xn = X[n, :].reshape(-1, 1); an = (xn.T @ W).reshape(1, -1)
    mun = np.exp(an - logsumexp(an)); mun[0, y[n]] -= 1.0; W = W - eta * xn @ mun
    ax = axs.flat[n]; ax.grid()
    z = lambda x: np.argmax(x @ W) + 1; zz = np.apply_along_axis(z, 1, XX)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyle='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 2, cmap='Blues'); plt.colorbar(cp, ax=ax, shrink=0.5)
    ax.arrow(0, 0, *W[:, y[n]], width=.03, facecolor='red', edgecolor='red')
    ax.scatter(*X.T, c=y, s=32); ax.scatter(X[n, 0], X[n, 1], facecolors='none', edgecolors='red', s=150)
```



Ejemplo de ingeniería de características: $C = 3$, $\mathbf{x} = (x_1, x_2)^t$

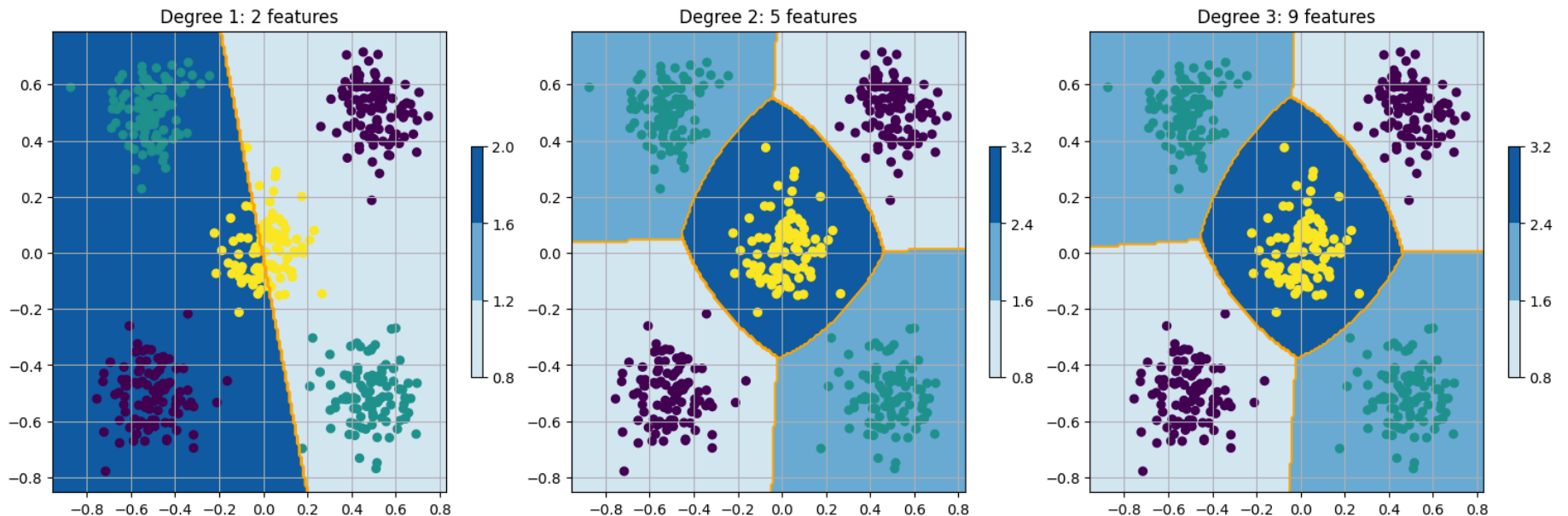
```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; import matplotlib.colors as mcol
from scipy.stats import multivariate_normal as mvn
from sklearn.preprocessing import PolynomialFeatures; from sklearn.linear_model import LogisticRegression
np.random.seed(234); N, S = 100, 0.01 * np.eye(2)
Gs = [mvn([0.5, 0.5], S), mvn([-0.5, -0.5], S), mvn([0.5, -0.5], S), mvn([-0.5, 0.5], S), mvn([0, 0], S)]
X = np.concatenate([G.rvs(size=N) for G in Gs])
y = np.concatenate((1 * np.ones(N), 1 * np.ones(N), 2 * np.ones(N), 2 * np.ones(N), 3 * np.ones(N)))
fig, ax = plt.subplots(figsize=(4, 4)); ax.grid(); ax.scatter(*X.T, c=y, s=32)
x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
XX = np.c_[np.ravel(xx), np.ravel(yy)]
```



Características polinómicas de grados 1, 2 y 3; sin sesgo:

$$\phi(\mathbf{x}) = (x_1, x_2)^t \quad \phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)^t \quad \phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3)^t$$

```
In [ ]: degrees = [1, 2, 3]; nrows, ncols = 1, 3
C=1e4 # C = 1 / lambda: varianza del prior
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 5), constrained_layout=True)
for i, degree in enumerate(degrees):
    ax = axes.flat[i]; ax.grid()
    transformer = PolynomialFeatures(degree, include_bias=False)
    X_poly = transformer.fit_transform(X)
    model = LogisticRegression(C=C, max_iter=1000).fit(X_poly, y)
    XX_poly = transformer.fit_transform(XX)
    zz = model.predict(XX_poly)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyle='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 2, cmap='Blues')
    plt.colorbar(cp, ax=ax, shrink=0.5)
    ax.scatter(*X.T, c=y, s=32)
    ax.set_title(f'Degree {degree}: {X_poly.shape[1]} features')
```



6 Regresión lineal

Regresión lineal: normal condicional para regresión, $y \in \mathbb{R}$,

$$p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mu, \sigma^2) \quad \text{con} \quad \mu = w_0 + \mathbf{w}^t \mathbf{x}$$

Notación homogénea: $\mathbf{w}$ absorbe el sesgo w_0

Regresión simple o múltiple: **simple** si la entrada es unidimensional; si no, **múltiple**

Regresión polinómica: $\mathbf{x}$ se procesa mediante un **extractor de características polinómico** ϕ

Extractor polinómico simple de grado D : $\phi(x) = (1, x, x^2, \dots, x^D)^t$

Regresión polinómica simple: para varianza σ^2 fija (regresión homocedástica)

$$p(y \mid x, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mu, \sigma^2) \quad \text{con} \quad \mu = \mathbf{w}^t \phi(x) = \sum_{d=0}^D w_d x^d$$

Estimador de Bayes: con pérdida cuadrática, $\ell_2(y - \pi(\mathbf{x})) = (y - \pi(\mathbf{x}))^2$, es la media a posteriori

$$\pi^*(\mathbf{x}) = \underset{\pi(\mathbf{x})}{\operatorname{argmin}} R(y \mid \mathbf{x}) = \underset{\pi(\mathbf{x})}{\operatorname{argmin}} \mathbb{E}_{p(y|\mathbf{x},\boldsymbol{\theta})}[(y - \pi(\mathbf{x}))^2] = \mathbb{E}[y \mid \mathbf{x}] = \mu$$

Riesgo de Bayes teórico ($\mathbf{w}$ conocido): es la varianza

$$R^*(y \mid \mathbf{x}) = \mathbb{E}_{p(y|\mathbf{x},\boldsymbol{\theta})}[(y - \pi^*(\mathbf{x}))^2] = \mathbb{E}_{p(y|\mathbf{x},\boldsymbol{\theta})}[(y - \mu)^2] = \sigma^2$$

Neg-log-verosimilitud: de $\theta = (\mathbf{w}^t, \sigma^2)^t$ respecto a N datos $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}$ no normalizada

$$\text{NLL}(\theta) = - \sum_{n=1}^N \log p(y_n \mid \mathbf{w}^t \mathbf{x}_n, \sigma^2) = \frac{N}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_n (y_n - \mathbf{w}^t \mathbf{x}_n)^2$$

Residual sum of squares (RSS): equivalente a la NLL para minimizar en $\mathbf{w}$, con $\mathbf{y} = (y_1, \dots, y_N)^t$

$$\text{RSS}(\mathbf{w}) = \sum_n (y_n - \mathbf{w}^t \mathbf{x}_n)^2 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 = (\mathbf{y} - \mathbf{X}\mathbf{w})^t (\mathbf{y} - \mathbf{X}\mathbf{w})$$

Gradiente (transpuesto):

$$\frac{\partial \text{RSS}(\mathbf{w})}{\partial \mathbf{w}} = 2 \sum_n (\mathbf{w}^t \mathbf{x}_n - y_n) \mathbf{x}_n^t = 2(\mathbf{X}\mathbf{w} - \mathbf{y})^t \mathbf{X}$$

Ordinary least squares (OLS):

$$\hat{\mathbf{w}} = \mathbf{X}^\dagger \mathbf{y} \quad \text{donde} \quad \mathbf{X}^\dagger = (\mathbf{X}^t \mathbf{X})^{-1} \mathbf{X}^t \text{ es la pseudoinversa izquierda de } \mathbf{X}$$

Error cuadrático medio de los residuos:

$$\hat{\sigma}^2 = \underset{\sigma^2}{\text{argmin}} \text{NLL}(\hat{\mathbf{w}}, \sigma^2) = \frac{1}{N} \sum_n (y_n - \hat{\mathbf{w}}^t \mathbf{x}_n)^2$$

Ejemplo: $p(y_n \mid x_n, m, c, \sigma^2) = \mathcal{N}(y_n \mid mx_n + c, \sigma^2)$, $\mathcal{D} = \{(x_n, y_n)\} = \{(0, 0), (1, 0.5), (1, 1.5), (2, 2)\}$

$$\mathbf{w} = (c, m)^t \quad \mathbf{X} = [1, 0; 1, 1; 1, 1; 1, 2] \quad \mathbf{y} = (0, 1/2, 3/2, 2)$$

$$\mathbf{X}^t \mathbf{X} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ 4 & 6 \end{pmatrix}$$

$$(\mathbf{X}^t \mathbf{X})^{-1} = \frac{1}{8} \begin{pmatrix} 6 & -4 \\ -4 & 4 \end{pmatrix} = \begin{pmatrix} 3/4 & -1/2 \\ -1/2 & 1/2 \end{pmatrix}$$

$$(\mathbf{X}^t \mathbf{X})^{-1} \mathbf{X}^t = \begin{pmatrix} 3/4 & -1/2 \\ -1/2 & 1/2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 3/4 & 1/4 & 1/4 & -1/4 \\ -1/2 & 0 & 0 & 1/2 \end{pmatrix}$$

$$\hat{\mathbf{w}} = (\mathbf{X}^t \mathbf{X})^{-1} \mathbf{X}^t \mathbf{y} = \begin{pmatrix} 3/4 & 1/4 & 1/4 & -1/4 \\ -1/2 & 0 & 0 & 1/2 \end{pmatrix} \begin{pmatrix} 0 \\ 1/2 \\ 3/2 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\mathbf{w}} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 2 \end{pmatrix}$$

$$\text{RSS}(\hat{\mathbf{w}}) = (\mathbf{y} - \hat{\mathbf{y}})^t (\mathbf{y} - \hat{\mathbf{y}}) = (0, 1/2, -1/2, 0)(0, 1/2, -1/2, 0)^t = 1/2$$

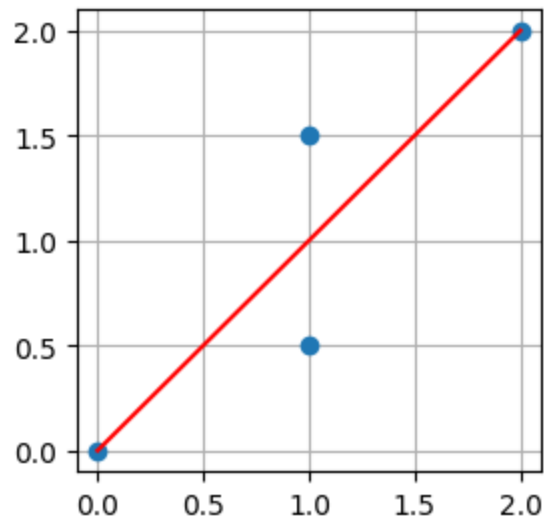
```
In [ ]: import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,0],[1,1],[1,1],[1,2]]); y = np.array([0,.5,1.5,2]); eta = .05; w = np.zeros(2)
for i in range(100):
    w = w - eta * 2.0*X.T@(X@w-y)
print(w)
```

```
[0.00 1.00]
```

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
X = np.array([0.0, 1.0, 1.0, 2.0]); N = len(X); Xh = np.c_[np.ones(N), X]
y = np.array([0.0, 0.5, 1.5, 2.0]); hw = np.linalg.inv(Xh.T @ Xh) @ Xh.T @ y
y_pred = Xh @ hw; RSS = np.square(y - y_pred).sum(); print(f'hw = {hw} RSS = {RSS}')
[m, c], [r], _, _ = np.linalg.lstsq(np.c_[X, np.ones(N)], y, rcond=None)
print(f'np.linalg.lstsq: hw = {hw} RSS = {RSS}')
fig = plt.figure(figsize=(3, 3)); plt.plot(X, y, 'o'); plt.plot(X, m*X + c, 'r'); plt.grid(True)
```

```
hw = [0. 1.] RSS = 0.5
```

```
np.linalg.lstsq: hw = [0. 1.] RSS = 0.5
```



Gráfica de residuos: si la entrada es unidimensional, evaluamos el modelo con $r_n = y_n - \hat{y}_n$ en función de x_n

Gráfica $\hat{y}_n$ vs y_n : si la entrada es multidimensional, evaluamos con $\hat{y}_n$ en función de y_n ; diagonal óptima

Residual sum of squares: medida de calidad obvia

$$\text{RSS}(\mathbf{w}) = \sum_n (y_n - \mathbf{w}^t \mathbf{x}_n)^2$$

Raíz del error cuadrático medio (RMSE): RSS normalizada

$$\text{RMSE}(\mathbf{w}) = \sqrt{\frac{1}{N} \text{RSS}(\mathbf{w})}$$

Suma de cuadrados total: suma de errores cuadráticos si siempre se predice la media empírica

$$\text{TSS}(\mathbf{w}) = \sum_n (y_n - \bar{y})^2 \quad \text{con} \quad \bar{y} = \frac{1}{N} \sum_n y_n$$

Coeficiente de determinación: varianza de las predicciones en relación con predecir siempre la media empírica

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$$

Interpretación de R^2 : cuanto más próximo a 1 sea, mayor será la reducción de la varianza y mejor el ajuste

Ejemplo: $f(x; \mathbf{w}) = w_0 + w_1x + w_2x^2$ con $w_0 = 0$, $w_1 = -1.5$ y $w_2 = 1/9 = 0.11$; $\sigma^2 = 4$

```
In [ ]: import numpy as np; np.random.seed(0); import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures; from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error as RMSE, r2_score as R2
w0 = 0; w1 = -1.5; w2 = 1/9; sigma = 2; f = lambda X: w0 + w1 * X + w2 * X * X
N = 21; X = np.linspace(0.0, 20, N); y = f(X) + np.random.normal(0, sigma, X.shape)
Xt = np.arange(0.0, 20, 0.1); yt = f(Xt) + np.random.normal(0, sigma, Xt.shape); degs = [1, 2, 3, 14]
fig, axs = plt.subplots(3, len(degs), figsize=(12, 4.5), constrained_layout=True)
for i, deg in enumerate(degs):
    poly = PolynomialFeatures(degree=deg, include_bias=True); Xp = poly.fit_transform(X.reshape(-1, 1))
    lr = LinearRegression(fit_intercept=False).fit(Xp, y); y_pred = lr.predict(Xp)
    Xtp = poly.transform(Xt.reshape(-1, 1)); yt_pred = lr.predict(Xtp)
    ax = axs.flat[i]; ax.grid(); ax.scatter(X, y, s=20); ax.plot(X, y_pred)
    ax.plot(Xt, f(Xt), color='orange'); ax.set_title(f'Deg {deg} D {Xp.shape[1]} {lr.coef_!s:.24s}', fontsize=10)
    ax = axs.flat[i+len(degs)]; ax.grid(); ax.set_ylim(-9, 9); ax.stem(X, y - lr.predict(Xp), markerfmt='.')
    ax.set_title(f'RMSE {RMSE(y, y_pred):.2f} (test {RMSE(yt, yt_pred):.2f})')
    ax = axs.flat[i+2*len(degs)]; ax.grid(); ax.set_ylim(-10, 15); ax.scatter(y, y_pred, s=20)
    ax.plot(y, y, color='orange'); ax.set_title(f'R2 {R2(y, y_pred):.2f} (test {R2(yt, yt_pred):.2f})')
```

